

Objetivo

Preparar una aplicación web Node.js sencilla y comprobar su funcionamiento local antes de contenerizarla. Construir una imagen Docker de la aplicación y cargarla en el clúster local de Minikube. Crear un Deployment mediante un manifiesto YAML para ejecutar la aplicación como un Pod administrado por Kubernetes. Crear un Service de tipo NodePort para exponer la aplicación y acceder a ella desde el navegador. Validar los recursos desplegados mediante kubectl y relacionar Pod, Deployment y Service dentro del flujo básico de Kubernetes.

Requisitos previos

- Tener Docker Desktop instalado y en ejecución.
- Tener Minikube instalado y disponible desde la terminal.
- Tener kubectl instalado y configurado.
- Tener Visual Studio Code instalado.
- Tener Git Bash instalado en Windows y configurado como terminal integrada de Visual Studio Code.
- Tener un navegador web moderno como Chrome, Edge o Firefox.
- Conocer los fundamentos básicos de Docker, contenedores y archivos YAML.

Duración estimada

60 minutos

Introducción

En esta práctica desplegarás una aplicación web Node.js en un clúster local de Kubernetes con Minikube. Primero crearás y validarás una página sencilla, después construirás su imagen Docker y la cargarás en Minikube. Finalmente definirás un Deployment y un Service NodePort mediante manifiestos YAML, comprobarás el estado de los recursos con kubectl y accederás a la aplicación desde el navegador. El objetivo es recorrer de principio a fin el flujo básico código → imagen → Deployment → Pod → Service → aplicación.

Instrucciones



Preparación del directorio de trabajo

En esta preparación crearás el directorio raíz del curso y el subdirectorio de la práctica 1. Esta estructura se conservará durante las prácticas siguientes para

mantener separados el código, las imágenes y los manifiestos de cada laboratorio.



Crear el directorio raíz y el subdirectorio de la práctica

- **Paso 1.** Abre **Docker Desktop** y espera a que el motor indique que está activo, porque Minikube utilizará Docker para ejecutar el nodo local del clúster.
- **Paso 2.** Abre **Visual Studio Code** y espera a que cargue completamente, ya que desde su terminal integrada crearás los archivos y administrarás Kubernetes.
- **Paso 3.** En VS Code, selecciona **Terminal** → **New Terminal** para abrir una consola integrada desde la cual ejecutarás todos los comandos requeridos.
- **Paso 4.** En la flecha desplegable situada junto al botón + del panel Terminal, selecciona **Git Bash** para trabajar con la sintaxis Bash utilizada en esta práctica.
- **Paso 5.** Ejecuta el comando siguiente para crear el directorio raíz del curso y el subdirectorio `lab1`, manteniendo los laboratorios organizados en una sola ubicación.

```
mkdir -p /c/LABS/kubernetes/lab1
```

[Copiar](#)

- **Paso 6.** Ejecuta el comando siguiente para abrir `C:\LABS\kubernetes` como carpeta de trabajo en VS Code y visualizar desde el Explorador los archivos del curso.

```
code /c/LABS/kubernetes
```

[Copiar](#)

- **Paso 7.** Si VS Code solicita confirmar la confianza de la carpeta, acepta la acción para habilitar la terminal integrada, el Explorador y la edición de archivos.
- **Paso 8.** En el Explorador de VS Code, confirma que aparezcan el directorio raíz `kubernetes` y el subdirectorio `lab1`, donde trabajarás durante esta práctica.

```
C:\LABS\kubernetes
```

 Copiar

```
└─ lab1
```

- **Paso 9.** Ejecuta el siguiente comando para cambiar la ubicación activa de Git Bash al directorio `lab1` y evitar crear archivos en una ruta incorrecta.

```
cd /c/LABS/kubernetes/lab1
```

 Copiar

- **Paso 10.** Ejecuta `pwd` para comprobar la ruta actual y confirma que la terminal se encuentre exactamente en `/c/LABS/kubernetes/lab1` antes de continuar.

```
pwd
```

 Copiar

Salida esperada:

Comprueba la salida obtenida y confirma que Git Bash se encuentra en el directorio destinado a la primera práctica.

```
/c/LABS/kubernetes/lab1
```

 Copiar

IMPORTANTE: En las prácticas siguientes conservarás `C:\LABS\kubernetes` y crearás únicamente el subdirectorio correspondiente a cada nuevo laboratorio.



Tarea 1. Crear y validar la aplicación Node.js

En esta tarea crearás una aplicación web sencilla con Node.js. La página mostrará información básica de la instancia que responde, permitiendo comprobar después que la aplicación realmente se está ejecutando dentro de un Pod de Kubernetes.

Tarea 1.1. Crear la estructura del proyecto

- **Paso 11.** Ejecuta el bloque siguiente para crear las carpetas `app` y `k8s`; la primera contendrá la aplicación y la segunda almacenará los manifiestos de Kubernetes.

```
mkdir -p app k8s
touch app/package.json app/server.js
app/index.html
touch Dockerfile .dockerignore
touch k8s/deployment.yaml k8s/service.yaml
```

 Copiar

- **Paso 12.** Ejecuta el comando siguiente para comprobar que los archivos fueron creados en las rutas esperadas antes de agregar código o configuración.

```
find . -maxdepth 2 -type f | sort
```

 Copiar

Salida esperada:

Comprueba que la estructura general corresponda con la referencia siguiente antes de continuar con la implementación.

```
./dockerignore
./Dockerfile
./app/index.html
./app/package.json
./app/server.js
./k8s/deployment.yaml
./k8s/service.yaml
```

 Copiar

Tarea 1.2. Crear la aplicación web

- **Paso 13.** Abre `app/package.json` desde el Explorador de VS Code y agrega la definición mínima del proyecto para iniciar el servidor mediante `npm start`.

```
{
  "name": "k8s-node-web",
  "version": "1.0.0",
  "description": "Aplicación Node.js para
laboratorio de Kubernetes",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "engines": {
    "node": ">=24"
  }
}
```

 Copiar

- **Paso 14.** Abre `app/server.js` y agrega el servidor HTTP que entregará la página principal y un endpoint `/health` utilizado para validar el estado de la aplicación.

```
const http = require('http');
const fs = require('fs');
const path = require('path');
const os = require('os');

const PORT = process.env.PORT || 3000;
const htmlPath = path.join(__dirname,
  'index.html');

const server = http.createServer((req, res) => {
  if (req.url === '/health') {
    res.writeHead(200, { 'Content-Type':
      'application/json' });
    return res.end(JSON.stringify({
      status: 'ok',
      pod: os.hostname()
    }));
  }

  if (req.url === '/') {
    const html = fs.readFileSync(htmlPath,
      'utf8')
      .replace('', os.hostname());

    res.writeHead(200, { 'Content-Type':
      'text/html; charset=utf-8' });
    return res.end(html);
  }

  res.writeHead(404, { 'Content-Type':
    'text/plain; charset=utf-8' });
  res.end('Recurso no encontrado');
});

server.listen(PORT, '0.0.0.0', () => {
  console.log(`Aplicación disponible en el
    puerto ${PORT}`);
  console.log(`Hostname: ${os.hostname()}`);
});
```

 Copiar

- **Paso 15.** Abre `app/index.html` y agrega la interfaz visual que permitirá reconocer la aplicación desplegada y observar el hostname de la instancia que responde.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <title>Kubernetes Node.js Lab</title>
  <style>
    * { box-sizing: border-box; }
    body {
      margin: 0;
      min-height: 100vh;
      display: grid;
      place-items: center;
      font-family: Arial, Helvetica, sans-serif;
      background: #f4f7fb;
      color: #172033;
    }
    .card {
      width: min(680px, 90%);
      padding: 36px;
      background: white;
      border: 1px solid #dfe6ef;
      border-radius: 16px;
      box-shadow: 0 12px 30px rgba(23, 32, 51,
.08);
    }
    .badge {
      display: inline-block;
      padding: 6px 10px;
      border-radius: 999px;
      background: #eef3ff;
      font-size: 14px;
      font-weight: 700;
    }
    h1 { margin-bottom: 10px; }
    p { line-height: 1.6; }
    code {
      display: inline-block;
      padding: 4px 7px;
      background: #f2f4f7;
      border-radius: 6px;
    }
    .status {
```

 Copiar

```
margin-top: 24px;
padding: 16px;
border-left: 4px solid #326ce5;
background: #f7f9fc;
}
</style>
</head>
<body>
  <main class="card">
    <span class="badge">Kubernetes Lab 1</span>
    <h1>Aplicación Node.js desplegada
correctamente</h1>
    <p>
      Esta página se entrega desde una
      aplicación Node.js empaquetada
      como contenedor y administrada por
      Kubernetes.
    </p>

    <div class="status">
      <strong>Instancia que respondió:</strong>
    <br>
      <code></code>
    </div>
  </main>
</body>
</html>
```

Tarea 1.3. Probar la aplicación localmente

- **Paso 16.** Ejecuta el servidor directamente con Node.js para comprobar primero que el código funciona correctamente antes de introducir Docker y Kubernetes en el flujo.

```
cd app
npm start
```

[Copiar](#)

Salida esperada:

Comprueba que Node.js confirme el puerto utilizado y que el proceso permanezca activo esperando solicitudes HTTP.

Aplicación disponible en el puerto 3000
Hostname: <nombre_del_equipo>

 Copiar

- **Paso 17.** Abre un navegador y visita `http://localhost:3000` para confirmar que la página se renderiza correctamente antes de empaquetar la aplicación.

`http://localhost:3000`

 Copiar

- **Paso 18.** Abre otra pestaña con `http://localhost:3000/health` para validar que el endpoint de comprobación devuelve un documento JSON con estado `ok`.

`http://localhost:3000/health`

 Copiar

Salida esperada aproximada:

```
{  
  "status": "ok",  
  "pod": "<nombre_del_equipo>"  
}
```

 Copiar

- **Paso 19.** Regresa a la terminal, presiona `Ctrl+C` para detener Node.js y vuelve al directorio raíz `lab1` antes de comenzar la construcción de la imagen.

`cd ..`

 Copiar

Tarea finalizada

Resultado esperado: Aplicación Node.js creada y validada localmente desde el navegador y el endpoint de salud antes de contenerizarla.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 1 con IA](#)



Tarea 2. Construir la imagen Docker y preparar Minikube

En esta tarea empaquetarás la aplicación como una imagen Docker y la cargarás explícitamente en Minikube. De esta forma Kubernetes podrá utilizar una imagen local sin depender de Docker Hub ni de otro registro externo.

Tarea 2.1. Crear los archivos de construcción

- **Paso 20.** Abre `.dockerignore` y agrega las exclusiones siguientes para impedir que archivos innecesarios del host formen parte del contexto utilizado por Docker.

```
.git
.gitignore
k8s
node_modules
npm-debug.log
*.md
```

 Copiar

- **Paso 21.** Abre `Dockerfile` y agrega la configuración siguiente para construir una imagen ligera basada en Node.js 24 LTS y ejecutar el proceso como usuario no root.

```
FROM node:24-alpine

WORKDIR /app

COPY app/package.json ./
COPY app/server.js ./
COPY app/index.html ./

ENV PORT=3000

EXPOSE 3000

USER node

CMD ["node", "server.js"]
```

 Copiar

Tarea 2.2. Construir y probar la imagen

- **Paso 22.** Ejecuta `docker build` desde la raíz de `lab1` para crear la imagen `k8s-node-web:1.0`, utilizando los archivos preparados en la tarea anterior.

```
docker build -t k8s-node-web:1.0 .
```

 Copiar

- **Paso 23.** Ejecuta el comando siguiente para comprobar que Docker registró la imagen con el repositorio y la etiqueta esperados antes de cargarla en Minikube.

```
docker images k8s-node-web:1.0
```

 Copiar

- **Paso 24.** Ejecuta temporalmente el contenedor en el puerto 3000 para validar que la imagen funciona de manera independiente antes de desplegarla en Kubernetes.

```
docker run --rm -d \
  --name k8s-node-web-test \
  -p 3000:3000 \
  k8s-node-web:1.0
```

 Copiar

- **Paso 25.** Ejecuta `curl` contra el endpoint `/health` para confirmar que el contenedor responde con HTTP 200 y que Node.js inició correctamente dentro de la imagen.

```
curl http://localhost:3000/health
```

 Copiar

Salida esperada aproximada:

```
{"status":"ok","pod":"<id_del_contenedor>"}
```

 Copiar

- **Paso 26.** Detén el contenedor temporal para liberar el puerto 3000 y conservar únicamente la imagen que será utilizada posteriormente por Kubernetes.

```
docker stop k8s-node-web-test
```

 Copiar

Tarea 2.3. Iniciar Minikube y cargar la imagen

- **Paso 27.** Ejecuta el comando siguiente para iniciar un clúster Minikube local con driver Docker y un nodo, suficiente para trabajar el flujo fundamental de esta práctica.

```
minikube start --driver=docker
```

 Copiar

- **Paso 28.** Ejecuta `kubectl get nodes` para confirmar que el nodo de Minikube alcanzó el estado `Ready` antes de intentar crear cualquier workload de la aplicación.

```
kubectl get nodes
```

 Copiar

Salida esperada aproximada:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	...	...

 Copiar

- **Paso 29.** Ejecuta `minikube image load` para copiar la imagen construida con Docker Desktop al almacenamiento de imágenes disponible para el nodo local de Minikube.

```
minikube image load k8s-node-web:1.0
```

 Copiar

- **Paso 30.** Ejecuta el comando siguiente para confirmar desde Minikube que la imagen local se encuentra disponible antes de crear el Deployment.

```
minikube image ls | grep k8s-node-web
```

 Copiar

Salida esperada aproximada:

```
docker.io/library/k8s-node-web:1.0
```

 Copiar

Tarea finalizada

Resultado esperado: Imagen Docker `k8s-node-web:1.0` construida, probada localmente y cargada correctamente en el clúster Minikube.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 2 con IA](#)

Tarea 3. Crear y desplegar la aplicación con un Deployment

En esta tarea definirás el estado deseado de la aplicación mediante un manifiesto YAML. Kubernetes utilizará el Deployment para crear y mantener el Pod necesario sin administrar directamente el contenedor.

Tarea 3.1. Crear el manifiesto `deployment.yaml`

- **Paso 31.** Abre `k8s/deployment.yaml` y agrega el manifiesto siguiente para declarar un Deployment con una réplica de la aplicación Node.js preparada previamente.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-web-deployment
  labels:
    app: node-web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: node-web
  template:
    metadata:
      labels:
        app: node-web
    spec:
      containers:
        - name: node-web
          image: k8s-node-web:1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 3000
              protocol: TCP
```

 Copiar

NOTA: Se utiliza `imagePullPolicy: Never` porque la imagen fue cargada manualmente en Minikube. En un entorno productivo normalmente se utilizaría una imagen almacenada en un registry accesible para los nodos.

Tarea 3.2. Validar el manifiesto antes de aplicarlo

- **Paso 32.** Ejecuta una validación local con `--dry-run=client` para comprobar la estructura del manifiesto sin crear todavía ningún recurso dentro del clúster.

```
kubectl apply \
  --dry-run=client \
  -f k8s/deployment.yaml
```

 Copiar

Salida esperada:

```
deployment.apps/node-web-deployment created (dry run)
```

 Copiar

- **Paso 33.** Ejecuta el comando siguiente para mostrar el manifiesto interpretado por kubectl y detectar errores básicos de indentación, nombres o estructura antes del despliegue.

```
kubectl apply \
  --dry-run=client \
  -f k8s/deployment.yaml \
  -o yaml
```

 Copiar

Tarea 3.3. Aplicar el Deployment y observar los recursos

- **Paso 34.** Ejecuta `kubectl apply` para enviar la declaración al API Server y permitir que Kubernetes cree los recursos requeridos para alcanzar el estado deseado.

```
kubectl apply -f k8s/deployment.yaml
```

 Copiar

- **Paso 35.** Ejecuta `kubectl rollout status` para esperar hasta que el Deployment confirme que la réplica solicitada se encuentra creada y disponible correctamente.

```
kubectl rollout status deployment/node-web-deployment
```

 Copiar

Salida esperada:

```
deployment "node-web-deployment" successfully rolled out
```

 Copiar

- **Paso 36.** Ejecuta los comandos siguientes para observar el Deployment, el ReplicaSet creado automáticamente y el Pod que ejecuta la aplicación dentro del clúster.

```
kubectl get deployments
kubectl get replicaset
kubectl get pods -l app=node-web -o wide
```

 Copiar

Resultado esperado:

El Deployment debe mostrar `1/1` disponible y el Pod asociado debe encontrarse en estado `Running` con `READY 1/1`.

- **Paso 37.** Ejecuta `kubectl describe` sobre el Deployment para revisar selector, estrategia, plantilla del Pod y eventos registrados durante la creación del workload.

```
kubectl describe deployment node-web-deployment
```

 Copiar

Tarea finalizada

Resultado esperado: Deployment `node-web-deployment` creado con una réplica disponible y un Pod en estado `Running` administrado por Kubernetes.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 3 con IA](#)

Tarea 4. Exponer la aplicación mediante un Service NodePort

En esta tarea crearás un Service que proporcione un punto estable de acceso hacia el Pod. La relación se realizará mediante la etiqueta `app: node-web`, introduciendo el funcionamiento básico de selectors sin profundizar todavía en su administración.

Tarea 4.1. Crear el manifiesto service.yaml

- **Paso 38.** Abre `k8s/service.yaml` y agrega el manifiesto siguiente para crear un Service NodePort que dirija las solicitudes recibidas hacia el puerto 3000 del Pod.

```
apiVersion: v1
kind: Service
metadata:
  name: node-web-service
  labels:
    app: node-web
spec:
  type: NodePort
  selector:
    app: node-web
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 3000
```

 Copiar

NOTA: Kubernetes asignará automáticamente el `nodePort` . No se fija manualmente para evitar conflictos con puertos utilizados por otros laboratorios.

Tarea 4.2. Validar y aplicar el Service

- **Paso 39.** Ejecuta la validación local del archivo `service.yaml` para comprobar que `kubectl` reconoce correctamente la definición antes de enviarla al API Server.

```
kubectl apply \
  --dry-run=client \
  -f k8s/service.yaml
```

 Copiar

Salida esperada:

```
service/node-web-service created (dry run)
```

 Copiar

- **Paso 40.** Ejecuta `kubectl apply` para crear el Service y establecer un punto de acceso estable hacia los Pods identificados mediante la etiqueta `app=node-web` .

```
kubectl apply -f k8s/service.yaml
```

 Copiar

- **Paso 41.** Ejecuta el comando siguiente para comprobar el tipo del Service, su ClusterIP y el puerto NodePort asignado automáticamente por Kubernetes.

```
kubectl get service node-web-service
```

 Copiar

Salida esperada aproximada:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
node-web-service	NodePort	10.96.xxx.xxx		80:3xxxx/TCP	<none> ...

 Copiar

Tarea 4.3. Acceder a la aplicación desde el navegador

- **Paso 42.** Ejecuta `minikube service` con la opción `--url` para obtener una dirección accesible desde el host sin depender de la topología interna utilizada por Minikube.

```
minikube service node-web-service --url
```

 Copiar

Salida esperada aproximada:

```
http://127.0.0.1:<puerto_asignado>
```

 Copiar

- **Paso 43.** Mantén activa la terminal si Minikube crea un túnel temporal y copia exactamente la URL mostrada para utilizarla durante las pruebas de navegador.

NOTA: Con el driver Docker en Windows, el proceso puede permanecer activo mientras mantiene el túnel. Utiliza otra terminal de VS Code para continuar con las validaciones.

- **Paso 44.** Abre la URL obtenida en Chrome, Edge o Firefox y confirma que aparezca la página **Aplicación Node.js desplegada correctamente**.
- **Paso 45.** Observa el valor mostrado en **Instancia que respondió** y comprueba que ahora corresponde al nombre del Pod, no al nombre del equipo Windows.

Tarea finalizada

Resultado esperado: Service `node-web-service` de tipo NodePort creado y aplicación Node.js accesible desde el navegador mediante la URL de Minikube.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 4 con IA](#)

✓ Tarea 5. Validar el flujo completo del despliegue

En esta tarea reunirás las comprobaciones esenciales del laboratorio para relacionar los recursos creados y verificar que Deployment, Pod y Service se encuentran operativos antes de continuar con la práctica complementaria.

Tarea 5.1. Revisar todos los recursos de la aplicación

- **Paso 46.** Ejecuta el comando siguiente para listar en una sola vista los recursos asociados con `app=node-web` y comprobar que Kubernetes mantiene el estado esperado.

```
kubectl get all -l app=node-web
```

 Copiar

Salida esperada aproximada:

```

NAME                                                    READY
STATUS    RESTARTS    AGE
pod/node-web-deployment-xxxxxxxx-xxxxx    1/1
Running    0           ...

NAME                                                    TYPE            CLUSTER-IP
EXTERNAL-IP    PORT(S)        AGE
service/node-web-service    NodePort        10.96.xxx.xxx
<none>         80:3xxx/TCP    ...

NAME                                                    READY    UP-TO-
DATE    AVAILABLE    AGE
deployment.apps/node-web-deployment    1/1      1
1           ...

NAME
DESIRED    CURRENT    READY    AGE
replicaset.apps/node-web-deployment-xxxxxxxx    1
1           1           ...

```

Tarea 5.2. Validar la relación entre Service y Pod

- **Paso 47.** Ejecuta el comando siguiente para comprobar los endpoints asociados con el Service y verificar que existe una dirección interna apuntando al puerto 3000 del Pod.

```
kubectl get endpoints node-web-service
```

Salida esperada aproximada:

```

NAME            ENDPOINTS            AGE
node-web-service    10.244.x.x:3000    ...

```

NOTA: En versiones recientes de Kubernetes puedes observar una advertencia indicando que el recurso `Endpoints` está deprecado. Para esta primera práctica se usa solo como comprobación sencilla; más adelante se trabajará con los recursos actuales de descubrimiento de endpoints.

Tarea 5.3. Consultar los logs de la aplicación

- **Paso 48.** Ejecuta el comando siguiente para recuperar los logs del contenedor administrado por el Deployment y confirmar que Node.js inició correctamente dentro del Pod.

```
kubectl logs deployment/node-web-deployment
```

 Copiar

Salida esperada aproximada:

```
Aplicación disponible en el puerto 3000  
Hostname: node-web-deployment-xxxxxxxxxx-xxxxx
```

 Copiar

Tarea 5.4. Ejecutar la verificación final

- **Paso 49.** Copia y ejecuta el bloque completo para comprobar automáticamente el nodo, Deployment, Pod, Service y endpoints antes de cerrar la práctica.

```
echo "=== Verificación final de la Práctica 1
=== "

NODE_READY=$(kubectl get node minikube -o
jsonpath='{.status.conditions[?
(@.type=="Ready")].status}')
if [ "$NODE_READY" = "True" ]; then
    echo "✅ Nodo Minikube en estado Ready"
else
    echo "❌ Nodo Minikube no está Ready"
fi

AVAILABLE=$(kubectl get deployment node-web-
deployment -o
jsonpath='{.status.availableReplicas}')
if [ "$AVAILABLE" = "1" ]; then
    echo "✅ Deployment disponible con 1 réplica"
else
    echo "❌ Deployment sin réplica disponible"
fi

POD_PHASE=$(kubectl get pods -l app=node-web -o
jsonpath='{.items[0].status.phase}')
if [ "$POD_PHASE" = "Running" ]; then
    echo "✅ Pod en estado Running"
else
    echo "❌ Pod no está en estado Running"
fi

SERVICE_TYPE=$(kubectl get service node-web-
service -o jsonpath='{.spec.type}')
if [ "$SERVICE_TYPE" = "NodePort" ]; then
    echo "✅ Service NodePort creado"
else
    echo "❌ Service NodePort no encontrado"
fi

ENDPOINT_COUNT=$(kubectl get endpoints node-web-
service \
-o jsonpath='{.subsets[0].addresses[*].ip}'
2>/dev/null | wc -w)

if [ "$ENDPOINT_COUNT" -ge "1" ] 2>/dev/null;
then
```

 Copiar

```
echo "✅ Service asociado con al menos un  
endpoint"  
else  
echo "❌ Service sin endpoint disponible"  
fi  
  
echo "=== Fin de verificación ==="
```

Salida esperada:

```
=== Verificación final de la Práctica 1 ===  
✅ Nodo Minikube en estado Ready  
✅ Deployment disponible con 1 réplica  
✅ Pod en estado Running  
✅ Service NodePort creado  
✅ Service asociado con al menos un endpoint  
=== Fin de verificación ===
```

[Copiar](#)

IMPORTANTE: No elimines el Deployment, el Service, la imagen ni el clúster Minikube. La **Práctica complementaria 1.1** utilizará estos recursos para trabajar validaciones con `kubectl` y manifiestos YAML.

Tarea finalizada

Resultado esperado: Deployment, ReplicaSet, Pod, Service y endpoint validados correctamente con `kubectl`, confirmando el flujo completo del despliegue.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 5 con IA](#)

Resolución de problemas

Problema 1. Minikube no inicia con el driver Docker

Síntoma: `minikube start --driver=docker` termina con un error relacionado con Docker o no crea el nodo `minikube`.

Causa probable: Docker Desktop no está activo, el motor todavía está inicializando o Minikube no puede comunicarse con el daemon local.

Solución:

Aplica las acciones siguientes en el orden presentado y confirma que Docker responda antes de intentar iniciar nuevamente el clúster.

```
docker info
minikube status
minikube start --driver=docker
```

[Copiar](#)

Si `docker info` falla, abre Docker Desktop y no continúes hasta que el motor responda correctamente.

Problema 2. El Pod muestra ErrImageNeverPull

Síntoma: El Pod no inicia y `kubectl get pods` muestra `ErrImageNeverPull` o un evento relacionado con la imagen `k8s-node-web:1.0`.

Causa probable: La imagen fue construida en Docker Desktop, pero no se cargó correctamente en el almacenamiento de imágenes utilizado por Minikube.

Solución:

Aplica los comandos siguientes para confirmar la imagen local, cargarla nuevamente en Minikube y recrear el Pod administrado por el Deployment.

```
docker images k8s-node-web:1.0
minikube image load k8s-node-web:1.0
minikube image ls | grep k8s-node-web
kubectl rollout restart deployment/node-web-
deployment
kubectl rollout status deployment/node-web-deployment
```

[Copiar](#)

Problema 3. El Pod permanece en Pending o no llega a Running

Síntoma: `kubectl get pods` muestra `Pending`, `ContainerCreating` durante demasiado tiempo o el Pod no alcanza `READY 1/1`.

Causa probable: El nodo no está listo, existe un problema con la imagen o Kubernetes registró un evento que impide iniciar el contenedor.

Solución:

Obtén el nombre del Pod y revisa su descripción para identificar el evento concreto generado por el scheduler, kubelet o runtime.

```
kubectl get nodes
kubectl get pods -l app=node-web
POD_NAME=$(kubectl get pods -l app=node-web -o
jsonpath='{.items[0].metadata.name}')
kubectl describe pod "$POD_NAME"
```

[Copiar](#)

Revisa especialmente la sección `Events` ubicada al final de la salida antes de realizar cualquier cambio adicional.

Problema 4. El Service existe pero la aplicación no responde

Síntoma: `kubectl get service node-web-service` muestra el Service, pero la URL obtenida con Minikube no carga la página.

Causa probable: El Service no encontró Pods compatibles, el Pod no está listo o el proceso de túnel de Minikube fue cerrado.

Solución:

Comprueba las etiquetas, los endpoints y el estado del Pod para determinar si el Service tiene un destino válido antes de volver a abrir la URL.

```
kubectl get pods -l app=node-web --show-labels
kubectl get endpoints node-web-service
kubectl get service node-web-service
minikube service node-web-service --url
```

[Copiar](#)

El Service debe disponer de al menos un endpoint con formato similar a

```
10.244.x.x:3000
```

Problema 5. El manifiesto YAML devuelve un error

Síntoma: `kubectl apply` muestra errores como `error parsing`, `mapping values are not allowed` o indica campos desconocidos.

Causa probable: La indentación cambió al editar el archivo, se utilizaron tabulaciones o alguna propiedad quedó ubicada en una sección incorrecta.

Solución:

Valida primero ambos archivos sin modificar el clúster y utiliza la salida detallada para localizar el manifiesto que contiene el problema.

```
kubectl apply --dry-run=client -f  
k8s/deployment.yaml  
kubectl apply --dry-run=client -f k8s/service.yaml
```

[Copiar](#)

Revisa los archivos en VS Code utilizando espacios para la indentación y vuelve a ejecutar la validación antes de aplicar los cambios.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica tendrás una aplicación web Node.js empaquetada como imagen Docker y desplegada en Kubernetes mediante un Deployment. La aplicación estará expuesta mediante un Service NodePort, será accesible desde el navegador y podrás identificar con `kubectl` la relación entre Deployment, ReplicaSet, Pod y Service.

Notas y consideraciones

- En Windows, ejecuta todos los comandos desde la terminal integrada de Visual Studio Code seleccionando el perfil Git Bash.
- Esta práctica utiliza un clúster Minikube de un nodo para concentrarse en el flujo fundamental de despliegue. Los escenarios multinodo se trabajarán posteriormente.
- No elimines el clúster ni los recursos al finalizar. La práctica complementaria 1.1 reutilizará este entorno para validar `kubectl` y los manifiestos YAML.

- La imagen se construirá con Docker Desktop y se cargará explícitamente en Minikube mediante `minikube image load` para evitar depender de un registro externo.

URL de referencia

- [Deployments](#)
- [Services](#)
- [kubectl](#)
- [Minikube](#)